The '~:' operation is shorthand to attach a label to a test. Its definition is shown below:

```
(~:) :: (Testable t) => String -> t -> Test
label ~: t = TestLabel label (test t)
```

By compiling and executing the above code with GHCi, you get the following result:

```
ghci> runTestTT test1
Cases: 2  Tried: 2  Errors: 0  Failures: 0
Counts {cases = 2, tried = 2, errors = 0, failures = 0}
```

A failure is reported when there is a mismatch between the expected result and the observed value. For example:

```
import Test.HUnit
test1 = TestList [ "Test addition" ~: 3 ~=? (2 + 1)
                 , "Test subtraction" ~: 3 ~=? (4 - 2)
                 ]
```

Running the above code reports the failure in the output:

```
ghci> runTestTT test1
### Failure in: 1:Test subtraction
expected: 3
 but got: 2
Cases: 2  Tried: 2  Errors: 0  Failures: 1
Counts {cases = 2, tried = 2, errors = 0, failures = 1}
```

If our test case definition is incorrect, the compiler will throw an error during compilation time itself! For example:

```
import Data.Char
import Test.HUnit
test1 = TestList [ "Test addition" ~: 3 ~=? (2 + 1)
                 , "Test case" ~: "EARTH" ~=? (map "earth")
                 ]
```

On compiling the above code, you get the following output:

```
ghci> :l error.hs

[1 of 1] Compiling Main             ( error.hs, interpreted )

error.hs:5:48:
    Couldn't match expected type `[Char]'
                with actual type `[a1] -> [b0]'
    In the return type of a call of `map'
    Probable cause: `map' is applied to too few arguments
    In the second argument of `(~=?)', namely `(map "earth")'
    In the second argument of `(~:)', namely
      `"EARTH" ~=? (map "earth")'
```

```
error.hs:5:52:
    Couldn't match expected type `a1 -> b0' with actual type
`[Char]'
    In the first argument of `map', namely `"earth"'
    In the second argument of `(~=?)', namely `(map "earth")'
    In the second argument of `(~:)', namely
      `"EARTH" ~=? (map "earth")'
Failed, modules loaded: none.
```

The correct version of the code and its output are shown below:

```
import Data.Char
import Test.HUnit
test1 = TestList [ "Test addition" ~: 3 ~=? (2 + 1)
                 , "Test case" ~: "EARTH" ~=? (map toUpper
"earth")
                 ]
```

The expected test output is as follows:

```
ghci> runTestTT test1
Cases: 2  Tried: 2  Errors: 0  Failures: 0
Counts {cases = 2, tried = 2, errors = 0, failures = 0}
```

In Haskell, one needs to check for an empty list when using the *head* function, or else it will throw an error. An example test is shown below:

```
import Test.HUnit
test1 = TestCase $ assertEqual "Head of emptylist" 1 (head [])
```

Executing the above code gives:

```
ghci> runTestTT test1
### Error:
Prelude.head: empty list
Cases: 1  Tried: 1  Errors: 1  Failures: 0
Counts {cases = 1, tried = 1, errors = 1, failures = 0}
```

Other than the *assertEqual* function, there are conditional assertion functions like *assertBool*, *assertString* and *assertFailure* that you can use. The *assertBool* function takes a string that is displayed if the assertion fails and there is a condition to assert. A couple of examples are shown below:

```
import Test.HUnit
test1 = TestCase $ assertBool "Does not happen" True
```

Executing the above code in GHCi gives you the following output:

```
ghci> runTestTT test1
Cases: 1  Tried: 1  Errors: 0  Failures: 0
```